B.M.S COLLEGE OF ENGINEERING

Department of Machine Learning

(B.E. Programme in AI & ML)

PROJECT REPORT – AAT-II

Course: Robotic Process Automation

Course Code: 24AM6PERPA

Project Title: Academic Plagiarism Detection Automation using UiPath

Student Name(s): Adithya Vipin , Amrith Thejas

USN(s): 1BM23AI009 , 1BM23AI019

Semester/Section: 6-A

Guide Name: Dr. Sandeep Varma N

Academic Year: 2025-2026 [Even Semester]

1. Introduction

This project implements an automated plagiarism detection system for academic submissions using UiPath Studio. The bot reads student-submitted documents (PDF/TXT), extracts text content, submits it to the Copyleaks plagiarism checking API via HTTP requests, retrieves the similarity report through a webhook mechanism, and logs results into an Excel sheet. Automated email alerts are triggered for high-similarity cases, notifying both the faculty and the concerned student, enabling quick identification and action on academic misconduct without manual intervention.

2. Problem Statement

Academic institutions face a growing challenge in ensuring the originality of student submissions. Manual plagiarism checking is time-consuming, inconsistent, and prone to human error. Faculty members spend significant time verifying each document individually using online tools, entering results manually, and following up with students. There is no automated, systematic workflow to handle this process at scale, leading to delays, missed cases, and administrative burden. The existing system lacks integration between document submission, verification, and reporting.

3. Objectives

- Automate the extraction of text from student-submitted PDF and TXT documents
- Integrate with the Copyleaks plagiarism detection API to check content similarity
- Automatically log similarity scores into a structured Excel report
- Send automated dual email alerts (to faculty and student) for submissions exceeding 30% similarity threshold
- Provide a recommendation based on the plagiarism score severity
- Implement robust error handling to ensure the bot continues even on partial failures
- Reduce manual effort and improve consistency in plagiarism review

4. Proposed Solution

The solution is a UiPath-based bot that monitors a designated folder for student submissions. Upon detecting files, the bot performs the following steps:

- Reads each document and extracts raw text using UiPath's Read PDF Text, Read PDF with OCR (fallback for scanned PDFs), or Read Text File activities
- Authenticates with the Copyleaks API using an API key and retrieves a Bearer access token
- Submits the extracted text to the Copyleaks scan endpoint via an HTTP PUT request, providing a webhook URL for asynchronous result delivery
- Waits 45 seconds for Copyleaks to process the scan and POST results to the webhook
- Polls the webhook.site endpoint via HTTP GET to retrieve the plagiarism result JSON

- Parses the JSON response to extract the aggregated plagiarism score (0-100)
- Generates a recommendation based on score thresholds (Reject / Manual Review / Minor Revision)
- Sends email alerts to both the professor and the student (looked up from a student database Excel file) if score exceeds 30%
- Logs results (student name, file name, score, date) into a structured Excel output file

5. Tools & Technologies

- UiPath Studio – Workflow design and bot development (VB.NET)
- Copyleaks API – Plagiarism detection and similarity scoring
- webhook.site – Webhook receiver for asynchronous API result delivery
- Gmail SMTP – Email notification automation (smtp.gmail.com, Port 587, StartTls)
- Microsoft Excel (.xlsx) – Student database and result logging
- Newtonsoft.Json – JSON parsing for API responses
- Tesseract OCR – Fallback OCR engine for scanned/image-based PDFs

6. System Architecture & Folder Structure

The project uses the following folder structure on the local machine:

- C:\RPA_Project\InputFiles\ – Student submission files (PDF/TXT)
- C:\RPA_Project\Output\Results.xlsx – Plagiarism results output
- C:\RPA_Project\ProcessedFiles\ – Processed files
- C:\RPA_Project\StudentDB.xlsx – Student name and email database

Input File Naming Convention: Files must follow the format: StudentName - AssignmentName.pdf

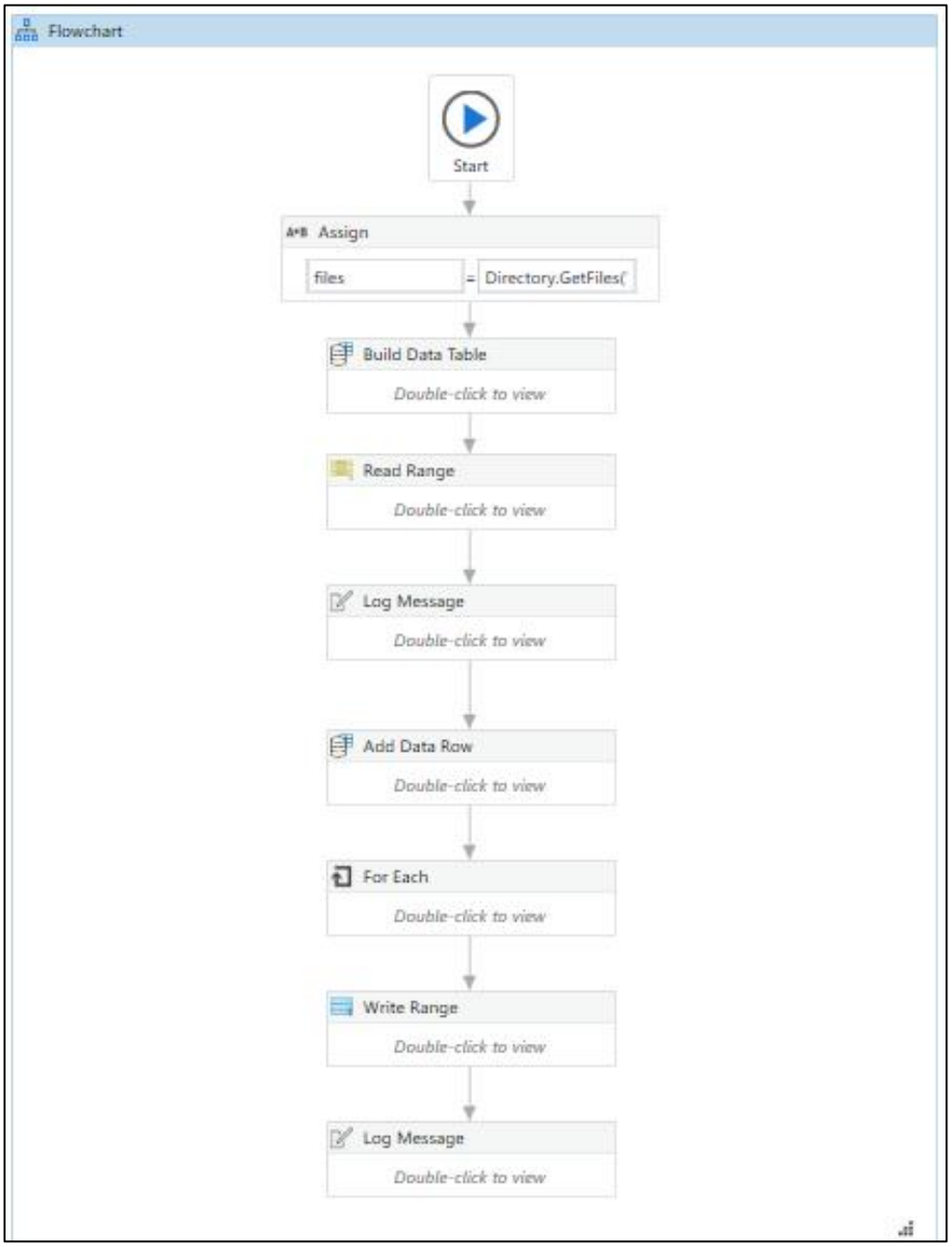
Example: Amrith Thejas - Space Exploration DOC_removed.pdf

Student Database (StudentDB.xlsx): Contains columns: SL NO | USN | STUDENT NAME | College Email ID. The bot performs a case-insensitive lookup using the STUDENT NAME column to retrieve the student's email.

7. Implementation

7.1 Main Flowchart

The overall flow consists of an initialization stage, a For Each file loop processing each submission, and a final output stage:



7.2 Initialization Stage

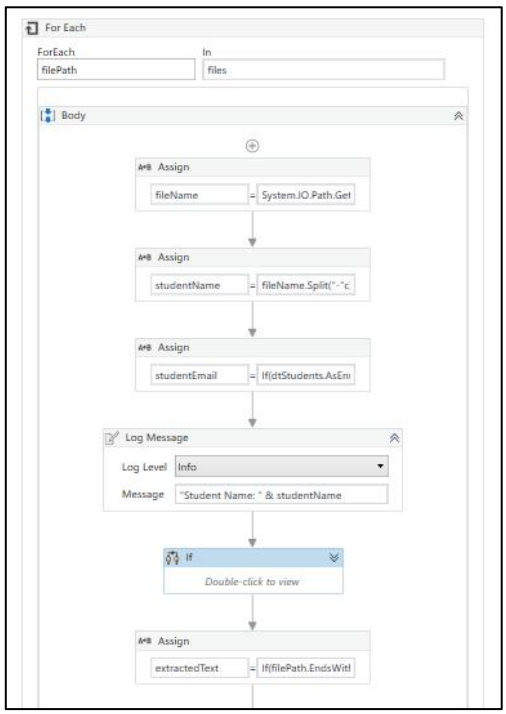
At startup, the bot performs the following:

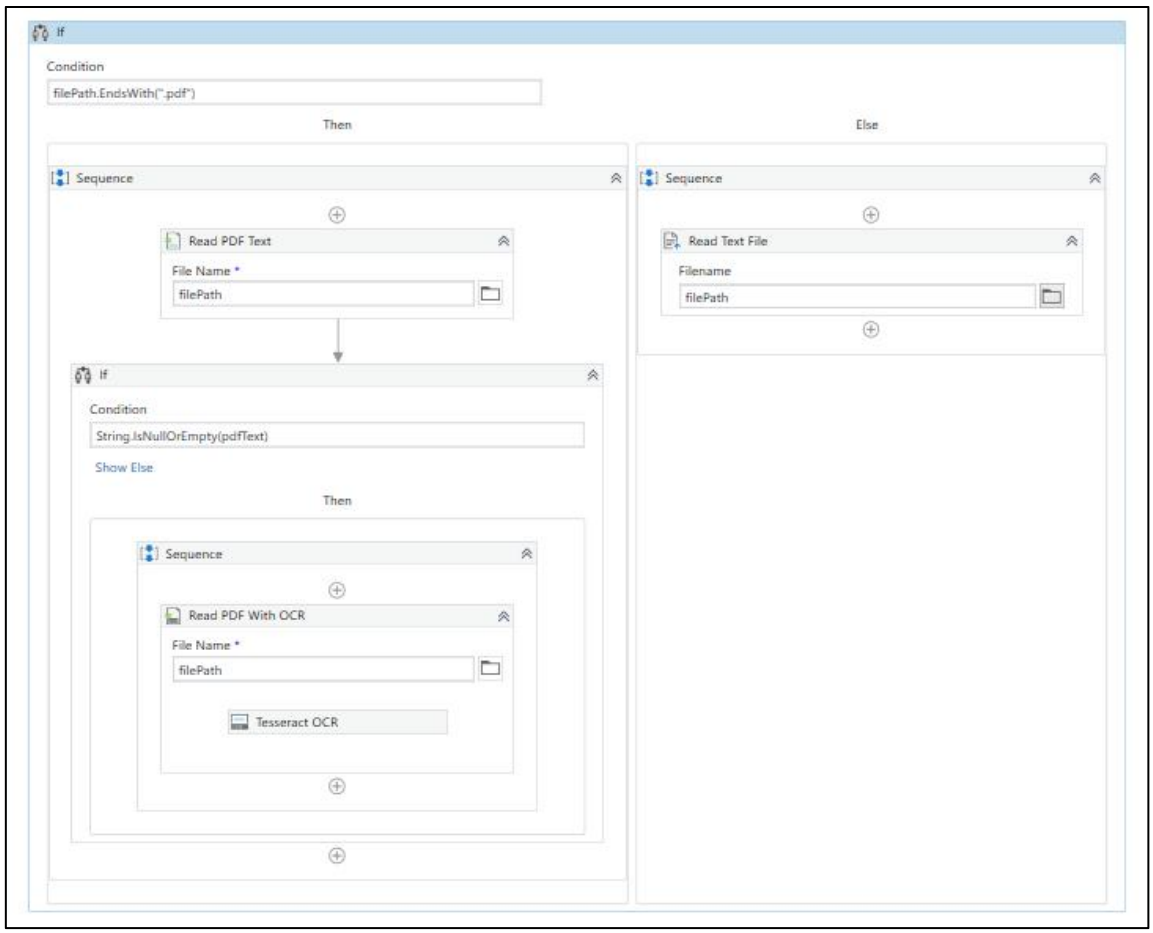
- Assigns all file paths from the InputFiles folder to the 'files' variable using Directory.GetFiles()
- Builds the output DataTable (dtResults) with columns: StudentName, FileName, PlagiarismScore, Date
- Reads StudentDB.xlsx into dtStudents for email lookups during processing
- Logs: 'PlagiarismDetectionBot Started. Total files found: X'

7.3 File Reading & Text Extraction

For each file, the bot first extracts the student name and looks up their email from the database, then reads the file content:

- Student name is extracted: fileName.Split("-"c)(0).Trim — takes the part before the first hyphen
- Email lookup performs a case-insensitive match: dtStudents.AsEnumerable().FirstOrDefault(Function(r) r("STUDENT NAME").ToString.ToLower = studentName.ToLower)
- PDF files: Read PDF Text activity; if the result is empty (scanned PDF), fallback to Read PDF With OCR using Tesseract engine
- TXT files: Read Text File activity
- extractedText is assigned based on file type using an If condition





7.4 Copyleaks API Integration

All three HTTP Request activities and the inner For Each are wrapped in a Try/Catch for error resilience. The three API calls are:

HTTP Request 1 (POST) – Copyleaks Login:

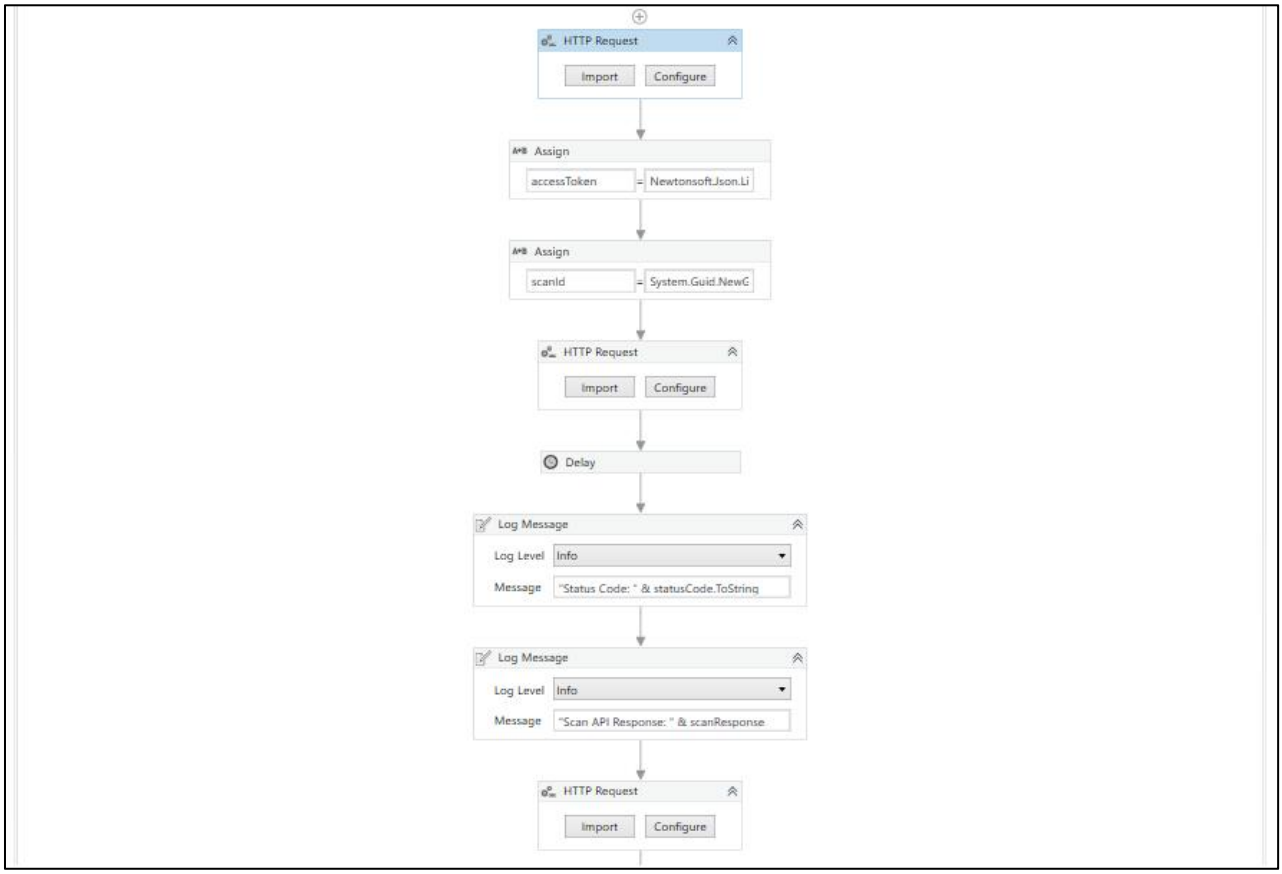
- URL: https://id.copyleaks.com/v3/account/login/api
- Body: { "email": "[email]", "key": "[API key]" }
- Response: accessToken extracted via JObject.Parse(responseString)("access_token").ToString

HTTP Request 2 (PUT) – Submit File for Scan:

- URL: https://api.copyleaks.com/v3/scans/submit/file/{scanId}
- Body: base64-encoded extractedText with webhook URL embedded in properties
- Headers: Authorization: Bearer [accessToken]
- Followed by a 45-second Delay activity to allow Copyleaks processing time

HTTP Request 3 (GET) – Poll Webhook:

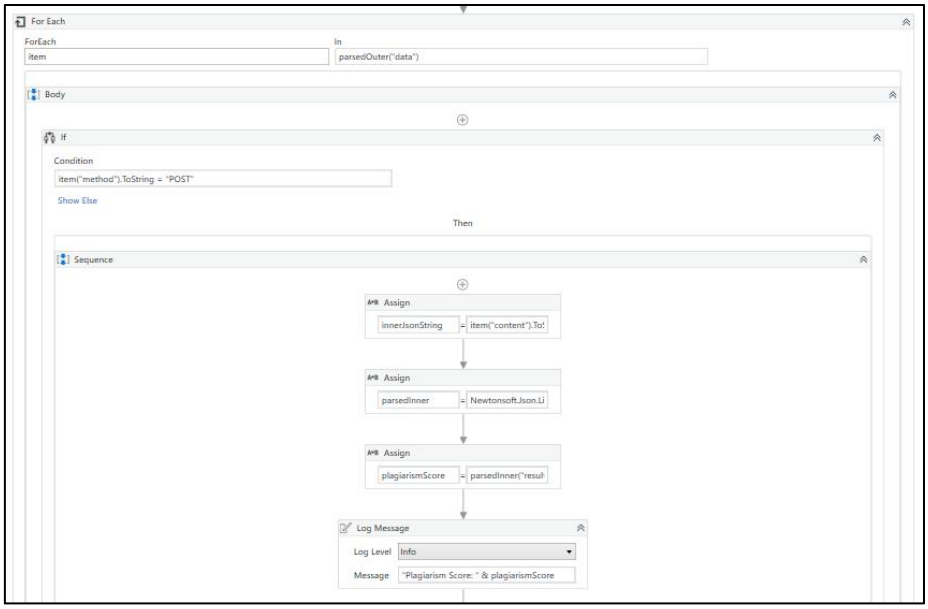
- URL: https://webhook.site/token/{UUID}/requests?sorting=newest&per_page=1
- Retrieves the Copyleaks result that was POSTed to the webhook URL



7.5 Result Parsing & Scoring

The webhook response is a JSON object. The bot iterates over the 'data' array using a For Each activity, looking for an item where item("method").ToString = "POST", then:

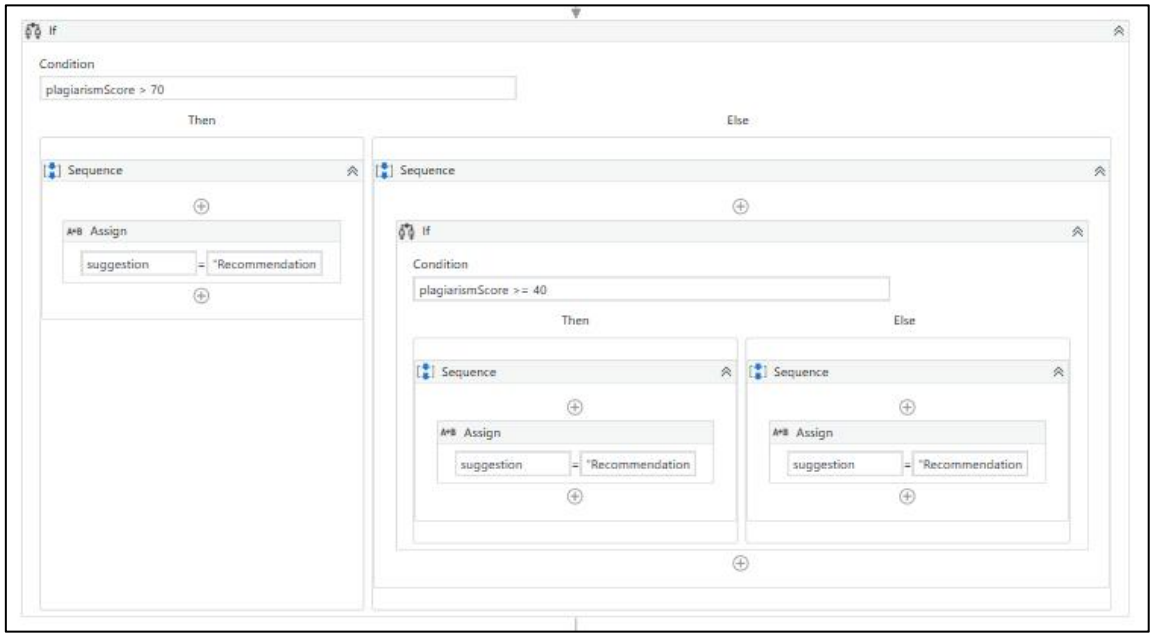
- Assigns innerJsonString = item("content").ToString
- Parses: parsedInner = JObject.Parse(innerJsonString)
- Extracts: plagiarismScore = parsedInner("results")("score")("aggregatedScore").ToObject(Of Double)
- Score is on a 0–100 scale (e.g. 100 = fully plagiarised)



7.6 Suggestion / Recommendation Logic

After extracting the score, a nested If/Else assigns a suggestion variable used in both email bodies:

- plagiarismScore > 70 → suggestion = "Recommendation: Reject submission immediately"
- plagiarismScore >= 40 → suggestion = "Recommendation: Manual review recommended"
- plagiarismScore < 40 → suggestion = "Recommendation: Minor revision required"
- plagiarismScore ≤ 30% → No email sent (below threshold)



7.7 Email Notification

If CDbl(plagiarismScore) > 0.3, two emails are sent via Gmail SMTP (smtp.gmail.com, Port 587, SecureConnection: StartTls):

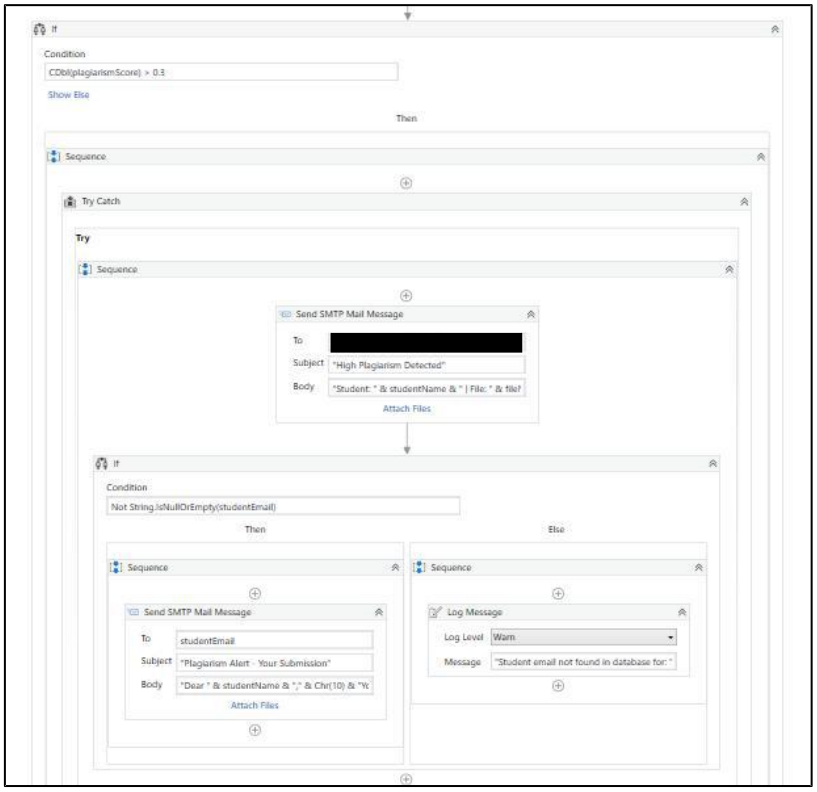
Email 1 – To Professor (collegeprofessor99@gmail.com):

- Subject: "High Plagiarism Detected"
- Body: "Student: " & studentName & " | File: " & fileName & " | Score: " & plagiarismScore.ToString("F1") & "%" & Chr(10) & suggestion

Email 2 – To Student (email from StudentDB):

- Subject: "Plagiarism Alert - Your Submission"
- Body: "Dear " & studentName & "," & Chr(10) & "Your submission '" & fileName & "' has been flagged." & Chr(10) & "Score: " & plagiarismScore.ToString("F1") & "%" & Chr(10) & suggestion
- If studentEmail is empty (student not found in DB), a Warning log is generated instead

Both SendMail activities are wrapped in a Try/Catch to handle SMTP failures without stopping the bot.



7.8 Excel Output

After processing all files, results are written to C:\RPA_Project\Output\Results.xlsx using Write Range activity. Each row contains: StudentName, FileName, PlagiarismScore, Date (dd-MM-yyyy format). The bot ends with a log message: 'PlagiarismDetectionBot Completed. Total files processed: X'.

8. Error Handling

The bot implements error handling at two levels:

Try/Catch – Copyleaks API:

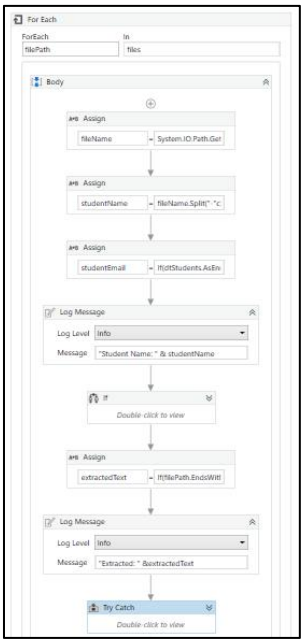
- Wraps all three HTTP Requests and the inner For Each loop
- On System.Exception: logs Error: "Copyleaks API failed for file: " & filePath & " - " & exception.Message
- Bot continues to the next file instead of crashing

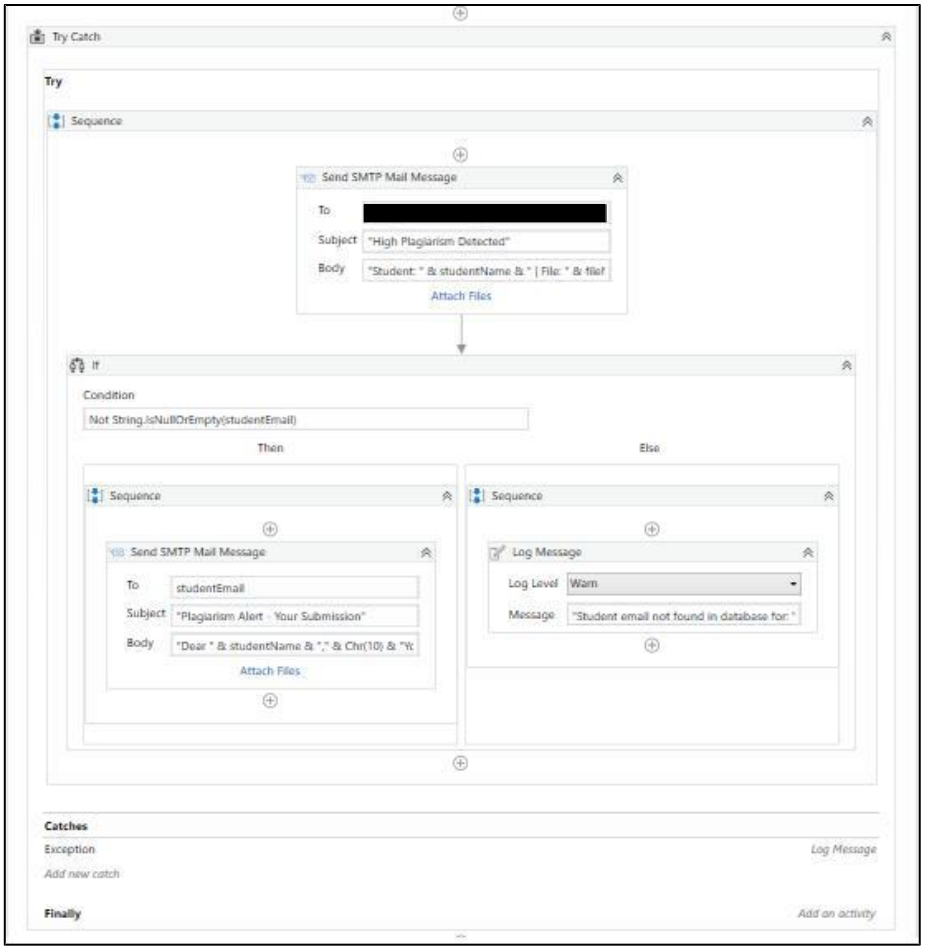
Try/Catch – Email Sending:

- Wraps both SendMail activities
- On System.Exception: logs Error: "Email sending failed: " & exception.Message

Empty Student Email Check:

- Before sending student email, checks: If Not String.IsNullOrEmpty(studentEmail)
- If empty: logs Warning: "Student email not found in database for: " & studentName





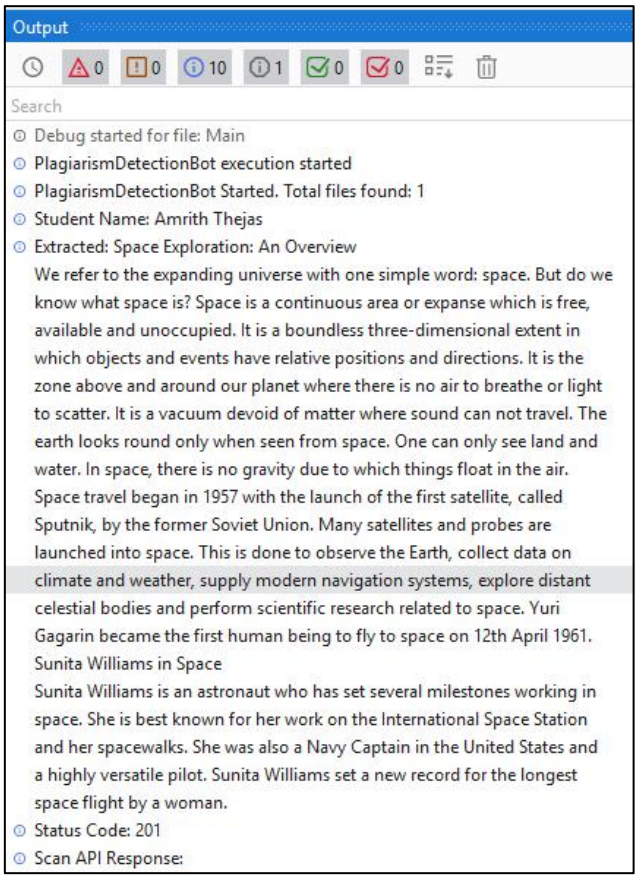
9. Expected Outcomes

- Significant reduction in faculty time spent on plagiarism checks (estimated 80% reduction)
- Consistent and objective similarity scoring across all submissions
- Automated Excel report with student-wise plagiarism scores ready for faculty review
- Timely dual email alerts (professor + student) ensuring high-risk cases are flagged immediately
- Intelligent recommendation guiding faculty on appropriate action per submission
- Enhanced academic integrity enforcement with minimal manual intervention

10. Results & Output

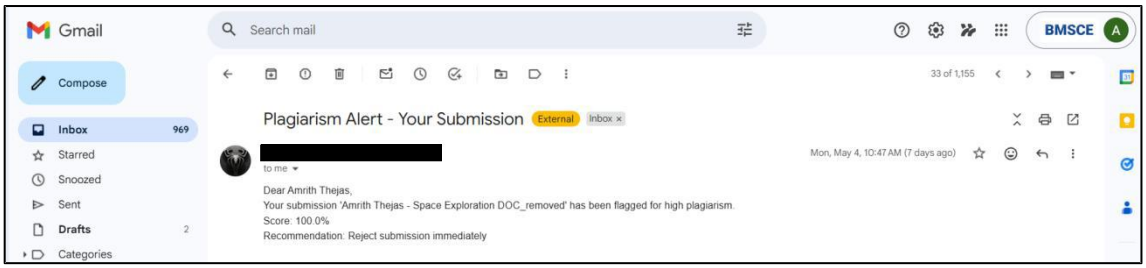
10.1 Plagiarism Score Detected

The bot successfully processed the test submission 'Amrith Thejas - Space Exploration DOC_removed.pdf' and detected a plagiarism score of 100.0% (a fully plagiarised document used for testing purposes). The UiPath output panel showed all log messages from start to completion.



10.2 Email to Professor

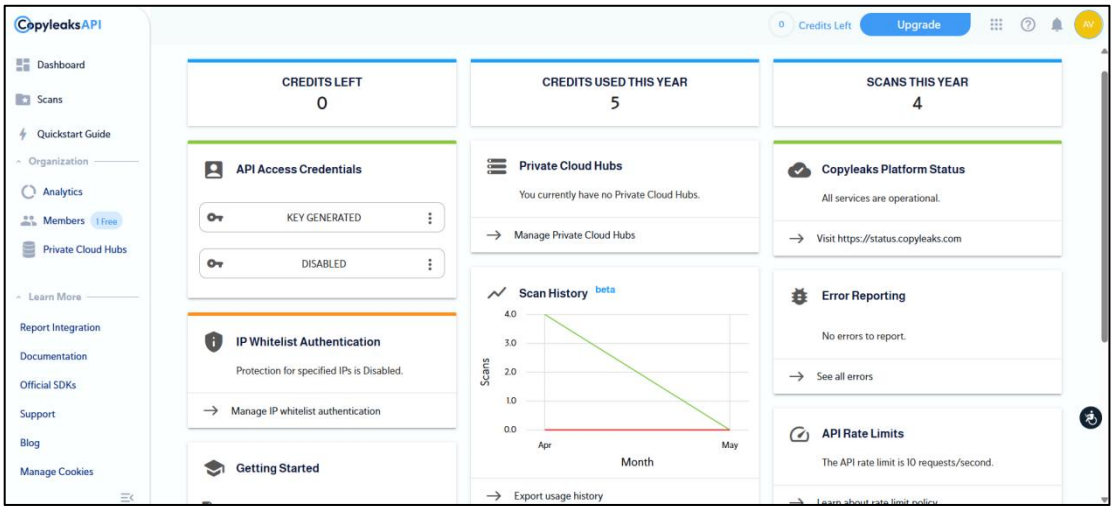
10.3 Email to Student



10.4 Excel Output (Results.xlsx)

	A	B	C	D
1	StudentName	FileName	PlagiarismScore	Date
2	Amrith Thejas	Amrith Thejas - Space Exploration D	96.7	11-05-2026
3				
4				

10.5 Copyleaks Dashboard



11. Challenges & Solutions

Challenge 1: Webhook URL expiry when the webhook.site tab becomes inactive.

Solution: Generated a new webhook.site UUID before each run and updated both HTTP Request activities accordingly.

Challenge 2: Copyleaks free account ran out of credits (5 credits per account).

Solution: Created a new Copyleaks account to get fresh credits and updated the API credentials in the login HTTP Request.

Challenge 3: Plagiarism score returning as 0.

Solution: Increased the Delay activity from 15 seconds to 45 seconds to give Copyleaks sufficient time to complete the scan before polling the webhook.

Challenge 4: Score displaying as 10000% in email body.

Solution: Removed the incorrect *100 multiplication. Copyleaks already returns the score as a 0–100 value.

Challenge 5: Gmail authentication failure (Error 535: Username and Password not accepted).

Solution: Regenerated Gmail App Password from Google Account → Security → App Passwords and updated both SendMail activities.

Challenge 6: VB.NET string concatenation compiler errors when using + operator.

Solution: Replaced + with & for string joining, which is the correct VB.NET operator.

Challenge 7: Student name case mismatch between filename and database (e.g. 'Amrith Thejas' vs 'AMRITH THEJAS').

Solution: Applied .ToLower() on both sides during the database lookup comparison to make it case-insensitive.

12. Timeline

Week 1: Design – Workflow planning, API key setup, folder structure configuration, student database creation

Week 2: Implementation – Bot development, Copyleaks API integration, webhook setup, Excel automation, email automation

Week 3: Testing – End-to-end testing with sample documents, debugging, edge case handling, error handling implementation

Week 4: Deployment – Final demo, documentation, report, and submission

13. Conclusion

The Academic Plagiarism Detection Automation bot successfully demonstrates the power of Robotic Process Automation in solving a real-world academic problem. By integrating UiPath Studio with the Copyleaks API, webhook.site, Gmail SMTP, and Excel, the bot provides a complete end-to-end automated solution that eliminates manual plagiarism checking, ensures consistent scoring, and delivers timely notifications to both faculty and students. The addition of a score-based recommendation system, a student database lookup, and robust two-level error handling further enhances the bot's practical utility and reliability in an academic environment.

Student Team Signature : Guide Name & Signature :

Student 1 : Adithya Vipin Dr. Sandeep Varma N

Student 2 : Amrith Thejas